A Parametric Drought-Index Insurance Simulator for Smallholder Maize Farmers in Eastern Rwanda

Technical Report, Final Version of the Product

Jok John Maker Kur, BSc Software Engineering, African Leadership University. Supervisor: Neza David Tuyishimire. 30 October 2026.

Resource	Link
Live dashboard	rdi-maize-insurance.vercel.app
API and interactive documentation	rdi-drought-api.onrender.com/docs
Source code	github.com/JokMaker/parametric-drought-simulator
Walkthrough video	Google Drive folder
USSD menu	`*384*44698#` in the Africa's Talking simulator (sandbox), or on the dashboard's USSD page

1. Introduction

This report describes the final version of my capstone project, a simulator for drought-index insurance for smallholder maize farmers in the seven districts of Eastern Rwanda, for Season A. In index insurance the payout is decided from data that anyone can check, instead of from a field visit after the harvest. Here the data are satellite rainfall from CHIRPS and crop greenness from MODIS NDVI. For each district I express both as anomalies against that district's own history, and a Ridge regression trained on NISR survey yields turns them into a predicted change in yield, which I call the index. A payout starts when the index falls to −2% and grows in a straight line until it is complete at −22%. Each payout is shown with its reason and its uncertainty, and the gap between what the index pays and what farmers actually lost (basis risk) is shown for every district.

The product has three parts. A Jupyter notebook does the data work and trains the model. A FastAPI backend (Python 3.12) serves the model, stores sign-ups and loss reports, and handles SMS and USSD through Africa's Talking. A Next.js 16 dashboard with 12 pages presents the results to analysts. Farmers do not need a smartphone: they can dial a USSD code from any phone, and the menu opens in Kinyarwanda.

2. Scope Alignment and Feature Validation

2.1 Planned features

The approved proposal (29 September 2026) listed ten functional requirements (FR1 to FR10) and six non-functional requirements (NFR1 to NFR6). The Phase column refers to the six phases of the proposal's Gantt chart (Section 1.10): 1 proposal and review, 2 data pipeline, 3 index construction, 4 calibration and model comparison, 5 dashboard and API, 6 backtesting, report and defence.

Planned feature	Phase	Intended purpose	Status	Evidence
FR1 Select a district and season and view drought risk	5	Let an analyst inspect any district-season	Completed	District Explorer; Drought Map with every season since 2001
FR2 Retrieve CHIRPS, MODIS NDVI and NISR data	2	Base the index on open data	Completed, changed	Notebook (Earth Engine extraction, 9 NISR releases), exported to backend/data
FR3 Anomaly score against each district's baseline	3	Make districts comparable	Completed	z-scores in the notebook, served by `GET /api/simulate`
FR4 Calibrate one pooled trigger and exit by regression	4	One payout rule that can be explained	Completed	Ridge, Lasso and quadratic OLS compared with nested LOOCV (notebook Sections 8 and 9); model.json
FR5 Simulate a historical payout	3, 5	Show what the cover would have paid	Completed	`IndexModel.payout()` in model.py; payout gauge in the Explorer
FR6 Show the payout with its reason	5	Transparency	Completed	`reason()` in payout.ts; Explorer and Farmer View
FR7 Show trigger and exit beside the anomalies	5	Show where a season sits on the rule	Completed	Explorer gauge; Overview
FR8 Show which input drives the decision	4, 5	Explainability	Completed	"What drives the index" on the Overview (greenness has 93% of the weight)

Planned feature	Phase	Intended purpose	Status	Evidence
FR9 Backtest against drought years and real yields	6	Check that the index is credible	Completed	Drought History: payouts in 34 of 63 drought-year seasons against 26 of 112 other seasons
FR10 Report basis risk per district	6	Disclose the main weakness	Completed	Basis Risk: from 33% in Nyagatare to 18% in Kayonza, with a trigger and exit simulator
NFR1 Fast response	5	Usable in normal conditions	Met	API median about 1 ms; 1,960 requests/s with 100 clients; every page under 2.5 s on a throttled phone (testing.md)
NFR2 No personal data	5	Privacy under Law 058/2021	Partly met, changed	Index data is district-level. USSD sign-up stores a phone number after the farmer confirms; sandbox and test numbers only (Section 2.3)
NFR3 Usable without statistics knowledge	5	Programme officers can read the results	Met	Plain titles, a guided tour, a reason on every payout; Lighthouse accessibility 100
NFR4 Portable with Docker	5	Same behaviour everywhere	Met	docker-compose.yml; the same Dockerfile runs on Render (render.yaml)
NFR5 Reproducible thresholds	4	Calibration can be audited	Met	Fixed inputs and seeds; tests check the API against the notebook's 175 index values
NFR6 Seven districts, Season A only	1	Keep the scope feasible	Met	Unchanged

2.2 Features added after the proposal

Most of these came from my supervisor's feedback. This Season follows the current season with NASA IMERG rainfall and daily MODIS data. Pricing works out premiums for each district. Field Checks catches losses the index misses. Farmer View shows a farmer's cover and can send the payout SMS. A USSD menu lets farmers check their cover, report a loss and sign up, and the Policies page lists everyone who has signed up.

2.3 Changes to the scope and why

Data. When I wrote the proposal I had only three usable years of yields, 21 district-seasons. I rebuilt 2024 by joining the raw Production and Screening files, and on 16 October my supervisor approved the NISR 2017 to 2021 releases, which brought the training set to 56 district-seasons from 2018 to 2025. I kept 2017 out of training because NISR imputed most of that season's production, and 2013 to 2016 have no harvest quantities. Yields rose by about 9.3% a year, so I removed that trend before computing anomalies. This mattered: an R^2 of 0.51 that I reported earlier for 2022 to 2025 came mostly from the trend. On all 56 seasons the leave-one-out R^2 is 0.11, and that is the figure I report.

Architecture. In the proposal the system called Google Earth Engine whenever a user asked for a district. That would have made responses slow (against NFR1) and needed cloud credentials on the server, so the notebook exports the results to files and a small script refreshes the current season. The computed results are small and do not change between runs, so they ship as versioned files instead of database tables, which also helps reproducibility (NFR5). Postgres, hosted on Neon, holds what users create: policies, loss reports and the SMS log.

Users and privacy. The proposal was written for analysts. My supervisor pointed out that the real users are Rwandan farmers, who should meet Kinyarwanda first and a simple code, so I added the USSD menu with sign-up. This departs from NFR2, because a policy has to be linked to a phone number. Sign-up only runs on the Africa's Talking sandbox and with demo numbers on the dashboard, and no real farmer's number has been collected. Before anything is saved, the confirmation screen tells the farmer that the number will be kept, and numbers are masked wherever they are displayed. Real use would first need ethics approval and compliance with Law 058/2021, including registration with the data-protection authority, written consent, retention limits and breach notification.

Moved to future work. An automatic payout run with mobile-money disbursement, an insurer login, early-warning SMS, live SMS with a registered sender ID (which needs a TIN) and a dedicated short code (which needs an allocation from RURA).

3. Technical Implementation

3.1 Architecture

The notebook pulls CHIRPS and MODIS through Google Earth Engine, reads the NISR microdata and writes `model.json`, `calibration.csv`, `seasons.csv`, `plot_losses.csv` and `current_season.json`. The backend loads these files once at start-up and works out the index, payout, price and basis risk for each request. The dashboard gets all its data from the backend as JSON. For USSD, Africa's Talking calls `POST /api/ussd` every

time the farmer presses a key, and the backend sends SMS through their messaging API. The dashboard runs on Vercel and the API on Render as a Docker container in Frankfurt, next to the Neon database. UptimeRobot calls `/api/health` every five minutes, which also keeps the free Render server awake. Both services redeploy when I push to `main`.

3.2 Main algorithms

1. Index and payout. For each district d, a variable x becomes an anomaly `z = (x − mean_d) / sd_d` over October to December. The index is `I = b0 + b1·z_rain + b2·z_ndvi` on standardised inputs, and the share of the cover that is paid is `P = min(1, max(0, (T − I) / (T − E)))`, with trigger T = −2 and exit E = −22.

2. Model comparison with nested leave-one-out cross-validation. Each of the 56 rows is held out once, and Ridge's α is tuned with a second leave-one-out loop on the other 55 rows only, so the held-out row has no influence on the tuning.

Model	RMSE	Predictive R^2
Ridge (selected)	22.26	0.11
Lasso	22.13	0.13
Quadratic OLS	22.69	0.08
Mean only (baseline)	24.09	−0.04

My proposal (Section 3.9) said I would choose the model with the lowest leave-one-out error, which would be Lasso, but I chose Ridge. The difference is 0.13 points of RMSE, about 0.6%, which I do not consider meaningful with 56 rows. Rainfall and greenness are correlated, and Lasso can drop one of them completely from one refit to the next, while Ridge keeps both with stable weights. That makes the index easier to explain and less likely to change when new seasons are added.

3. Trigger search. I tried every whole-number trigger from −1% to −30% on the leave-one-out index and scored each by its hit rate minus its false payout rate, with the exit 20 points below the trigger. The best was −2%, with the exit at −22%. It pays in 50% of the poor seasons and in 22% of the seasons that were not poor, the trade-off research question 2 asked about.

4. Uncertainty. I treat the leave-one-out RMSE of 22.3 points as a normal error, so the probability that the real harvest fell below the trigger is `Φ((T − I) / 22.3)`, with the normal CDF computed by a rational approximation.

5. Pricing. A field's value is area × expected yield × the farm-gate price of RWF 200/kg, and the cover is 22% of that. The premium is the cover × the district's average payout over 25 seasons (burning cost) × 1.3 for loading × 0.6 for a 40% subsidy. The same calculation exists in Python (`CoverQuote`) and TypeScript (pricing.ts), and a test checks that both quote RWF 584 for policy RDI-RWA-1233.

6. Field-check rule. A district needs a field check when the payout is under 25% but at least 3 farmers have reported a loss, or at least 45% of the farms NISR surveyed harvested less than half the expected yield. Season A 2019 flags all seven districts.

7. USSD without server state. Africa's Talking sends the whole session on every request as one string, for example `5*1*3*1250`, so the server does not need to remember anything. `Session.replay()` in ussd.py walks through the string: 5 on the main menu switches the language, and 0 goes back one screen. Sign-up reads the remaining answers in order and skips invalid ones, so a wrong answer repeats the question instead of ending the session.

8. SMS length. `sms_parts()` counts GSM-7 characters: 160 for one SMS and 153 per part when split, with extension characters counting twice. One character outside GSM-7 switches the message to UCS-2, where the limit is 70. In live mode the gateway refuses messages that would be split and stops at a daily limit.

9. Phone matching. Numbers are compared on their last nine digits, so `+250788…` and `0788…` count as the same farmer, and the database allows one policy per number.

10. Season monitor reliability. I re-ran the provisional index on 25 past seasons at each date. It agrees with the final payout decision 60% of the time by mid-October and 78% at the close of the season, and the dashboard shows this next to the provisional value.

3.3 Handling bad input

The API returns 404 for an unknown district, 422 for an invalid policy number or season, 503 when SMS is not configured, and 502 with the reason when the SMS provider rejects a message. On the dashboard, number fields keep what the user types and only correct it when they leave the field, so a field size of 10 m² becomes the 50 m²

minimum. In USSD an invalid answer repeats the question. If the database cannot be reached, a request fails after 8 seconds instead of 30, because Africa's Talking ends a USSD session after about 10 seconds.

4. Code Quality and Object-Oriented Design

4.1 Structure

The backend is split into small modules: four routers (`index`, `farmers`, `policies`, `messaging`), model.py with `IndexModel`, data.py with `Dataset`, policies.py with `Policy`, `PolicyBook` and `CoverQuote`, reports.py with the database store classes, messaging.py with the SMS gateways, ussd.py with `Session` and `UssdMenu`, and dependencies.py, which creates these objects and passes them to the routes.

4.2 Object-oriented principles

Principle	Where it is used
Encapsulation	Routes call `store.register()`, `dataset.result()` or `gateway.send()` and never see SQL, file layouts or HTTP
Abstraction and polymorphism	`SmsGateway` is abstract, with `UnconfiguredGateway`, `SandboxGateway` and `LiveGateway`; `ReportStore` is abstract, with `SqliteStore` and `PostgresStore`. The calling code does not know which one it has
Inheritance	`AfricasTalkingGateway` holds the shared request, and `LiveGateway` overrides `check()` and `fields()` to add the daily limit and sender ID. Every query is written once in `ReportStore`; the subclasses only open the connection
Single responsibility	Each class has one reason to change: `IndexModel` the formula, `Dataset` the data files, `CoverQuote` the price, `UssdMenu` the screens, the stores the database
Open/closed and Liskov substitution	A new database or SMS provider is a new subclass. `SqliteStore` and `PostgresStore` pass the same tests, so either can replace the other without changing the routes
Dependency inversion	Routes receive the dataset, store, gateway and menu through FastAPI's `Depends`, and the gateway takes its HTTP transport as a parameter, so tests can swap each one
Immutable values	`SeasonResult`, `Policy`, `CoverQuote`, `SmsSettings` and `Session` are frozen dataclasses
Separation of concerns (frontend)	Calculations are plain TypeScript modules in `lib/` (`payout`, `pricing`, `fieldChecks`, `sms`), separate from the components that display them; pages share components such as `Card`, `StatStrip`, `NumberInput` and `DistrictMap`

4.3 Practices

The README documents the setup and structure. Python uses `snake_case` and the JSON API uses `camelCase`, converted by a shared `CamelModel`. Every module has a docstring, secrets are only kept in environment variables, and the repository has 70 commits since 28 September, each for one change.

5. Testing and Performance

5.1 Strategies and results

Full results and reproduction steps are in docs/testing.md.

Strategy	Why	How	Result
Unit and API tests (pytest)	The maths and endpoints must match the notebook	71 tests, plus 2 against a temporary Postgres schema; a temporary database and a fake SMS gateway are injected through dependency overrides	73 of 73 pass, on SQLite and Postgres
Frontend unit tests (Node test runner)	Pricing, payouts, uncertainty and SMS length are calculated in the browser	33 tests, including a check that the Pricing page and the USSD menu quote the same premium	33 of 33 pass
Functional browser tests (check.mjs)	Check real user flows with valid and invalid input	Headless Chrome through the DevTools protocol, 7 flows: policy lookup in lower case, an unknown policy, the area field, SMS in two languages, USSD navigation, USSD sign-up with a wrong district, pricing	7 of 7 pass on the live site
Compatibility (webdriver.mjs)	Phones in Rwanda use Chromium or WebKit browsers	12 pages at phone, tablet and desktop size, light and dark, in Chrome 156; all pages at three widths in Firefox 159 and Safari 26	Chrome 72/72, Firefox 36/36, Safari 36/36

Strategy	Why	How	Result
Device testing	Farmers mostly use budget Android phones	The live site by hand on a Tecno Camon Slim (Android, Chrome, Wi-Fi), including rotation; a low-end phone emulated with Slow 4G and a 4× slower CPU	All pages work and fit the screen; about 1 to 2 s per page on Wi-Fi
Performance and load (benchmark.py)	A USSD reply must arrive within about 10 s, and rural connections are slow	API benchmark and load test; throttled page loads; Lighthouse (mobile); live timings from Kigali	API median about 1 ms; 1,960 requests/s with 100 clients and no errors; every page within 2.49 s; Lighthouse 93 to 97 for performance, 100 for the rest; USSD screen in 0.43 s; 37 MB of memory idle, 45 MB under load
Model validation	The index's accuracy should not be overstated	Nested LOOCV, leave-one-season-out, drought-year backtest	R^2 0.11 (leave-one-out), −0.11 for an unseen season; drought years paid 2.3 times as often as other years

5.2 Fixes made because of testing

On a 360-pixel phone the Overview scrolled sideways by 29 pixels, so the chart labels now sit above the bars on small screens. The season strip lost the selected season when a phone was rotated, and now re-centres when its width changes. Lighthouse found text with too little contrast (as low as 2.8:1); after changing the colours every text element passes the WCAG AA ratio of 4.5:1. Deployment found more: a setting pasted into Render with a trailing line break broke the database connection, so settings are now trimmed; the uptime monitor's HEAD requests were refused, so the health check accepts them; USSD SMS went to the demo phone instead of the caller; and a database outage could keep a USSD session waiting for 30 seconds, which I cut to 8.

5.3 Limits of the testing

The real-phone check was done by hand on one phone over Wi-Fi, so load times on a slow mobile connection were only measured in emulation. I have not yet run usability sessions with farmers, and the Kinyarwanda text still needs a native speaker's review.

6. Conclusion

All ten functional requirements are implemented and deployed, and five of the six non-functional requirements are met; NFR2 changed for the reasons in Section 2.3. For the research questions, the index has a real but weak relationship with yield (leave-one-out R^2 0.11), a single pooled rule pays in half of the poor seasons and in 22% of normal seasons, and Ridge and Lasso perform about the same, both better than quadratic OLS and the baseline. Basis risk is still the main weakness, which is why it is shown for every district and why the field-check route exists. The next steps are ethics approval, a native speaker's review of the Kinyarwanda, testing with farmers on their own phones, partnering with a licensed insurer, a payout run with mobile-money disbursement, live SMS, and re-calibration as NISR publishes new seasons.

References

Funk, C., Peterson, P., Landsfeld, M., Pedreros, D., Verdin, J., Shukla, S., Husak, G., Rowland, J., Harrison, L., Hoell, A., & Michaelsen, J. (2015). The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data, 2*, 150066. https://doi.org/10.1038/sdata.2015.66

Mazviona, B., & Sølvsten, S. (2025). Can index insurance keep up with climate change? Rethinking historical data models. *Frontiers in Climate, 7*, 1649540. https://doi.org/10.3389/fclim.2025.1649540

National Institute of Statistics of Rwanda (NISR). (2018–2025). *Seasonal Agricultural Survey* reports and microdata.

Republic of Rwanda. (2021). Law N° 058/2021 of 13/10/2021 relating to the protection of personal data and privacy. *Official Gazette of the Republic of Rwanda*.

Ryo, M. (2022). Explainable artificial intelligence and interpretable machine learning for agricultural data analysis. *Artificial Intelligence in Agriculture, 6*, 257–265.

Stigler, M., & Lobell, D. (2024). Optimal index insurance and basis risk decomposition: An application to Kenya. *American Journal of Agricultural Economics, 106*(1), 306–329. https://doi.org/10.1111/ajae.12375

Turvey, C. G., & McLaurin, M. K. (2012). Applicability of the Normalized Difference Vegetation Index (NDVI) in index-based crop insurance design. *Weather, Climate, and Society, 4*(4), 271–284. https://doi.org/10.1175/WCAS-D-11-00059.1